

P2308R0: Template parameter initialization

Audience: CWG

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

February 10, 2023

History

R0: issue drafting converted to a paper after initial review.

Introduction

[CWG2459](#) points out that there is no specification for how template parameters are initialized beyond “conversion to the type of the template-parameter” ([temp.type]/1.3) and “a converted constant expression ([expr.const]) of the type of the *template-parameter*” ([temp.arg.nontype]/2). As originally reported by Richard Smith, with template parameters of class type that have lvalue template parameter *objects* whose addresses can be examined during [non-trivial construction](#) and for which “[converted constant expression](#)” is [inadequate](#), this becomes an acute concern. This paper resolves that issue, along with [CWG2450](#) (including an extension of [Jens Maurer’s drafting](#)) and [CWG2049](#) which extend the set of potential initializers.

Approach

To avoid address-based paradoxes, template arguments for a template parameter of class type *C* are used to first initialize a temporary of that type called an *exemplar*. No restrictions are imposed on the conversion from a template argument to a constructor parameter, since `explicit` and list-initialization may already be used to limit conversions in a similar fashion. Each exemplar is used to copy-initialize the template parameter object to which it is (to be) template-argument-equivalent; the initialization is required to produce a template-argument-equivalent value. The multiple initializations of the template parameter object are (required to be) all equivalent and produce no side effects, so it is unobservable which happen. Using the same copy constructor for the exemplar (when the template argument is a template parameter object) and for initializing the template parameter object guarantees that the same value will be selected, as required for deduction, specialization, and the notion of the current specialization to make sense.

Wording

Relative to N4928.

`#[dcl.dcl]`

`#[dcl.fct.default]`

Change paragraph 3:

- A default argument shall be specified only in the *parameter-declaration-clause* of a function declaration or *lambda-declarator* or in a *template-parameter* ([temp.param]); ~~in the latter case, the initializer-clause shall be an assignment-expression~~. [...]

#[dcl.init.list]

Insert before bullet (1.6):

- as a template argument ([temp.arg.nontype])

#[temp]

Replace all uses *template-argument* in [temp.param]/12–16 with “template argument”.

#[temp.names]

Change the grammar in paragraph 1:

- [...]

template-argument:

constant-expression

type-id

id-expression

braced-init-list

#[temp.arg]

#[temp.arg.general]

Change paragraph 4:

- [...]

For a ~~template-argument~~ template argument that is a class type or a class template, the template definition has no special access rights to the members of the ~~template-argument~~ template argument.

[Example:

[...]

— end example]

Change paragraph 5:

- When template argument packs or default ~~template-argument~~ template arguments are used, a *template-argument* list can be empty. [...]

Change paragraph 7:

- If the use of a ~~template-argument~~ template argument gives rise to an ill-formed construct in the instantiation of a template specialization, the program is ill-formed.

#[temp.arg.nontype]

Change paragraph 1:

- If the type T of a *template-parameter* ([temp.param]) contains a placeholder type ([dcl.spec.auto]) or a placeholder for a deduced class type ([dcl.type.class.deduct]), the type of the parameter is the type deduced for the variable x in the invented declaration

$T \ x = \text{template-argument } E ;$

where E is the template argument provided for the parameter.

[Note: E is a *template-argument* or (for a default template argument) *initializer-clause*. — end note]

If a deduced parameter type is not permitted for a *template-parameter* declaration ([temp.param]), the program is ill-formed.

Insert before paragraph 2:

- When considering a template argument A for a non-type *template-parameter* of (possibly deduced) type T , a glvalue *exemplar* expression of type T , or of the referenced type if T is a reference type, is selected. The result of the exemplar determines the value of the *template-parameter* and the identity of the specialization ([temp.type]).

Change paragraph 2:

- ~~A template-argument for a non-type template-parameter~~ If T is not a class type and A is not a braced-init-list, A shall be a converted constant expression ([expr.const]) of the type of the template-parameter T ; the exemplar is A (as converted).

~~[Note: If the template-argument is an overload set (or the address of such, including forming a pointer-to-member), the matching function is selected from the set ([over.over]). — end note]~~

Insert before paragraph 3:

- Otherwise, a temporary variable

$\text{const } T \ v = A;$

is introduced. If the full-expression of the *init-declarator* for v is not a constant expression when interpreted as a *constant-expression* ([expr.const]), the program is ill-formed. The lifetime of v ends immediately after initializing it and any template parameter object (see below). The exemplar is v .

- If T is a class type, the value of the exemplar determines the identity of the template parameter object for the parameter ([temp.param]). Each template parameter object P is copy-initialized from an unspecified exemplar whose value nominates P . If, for any such initialization:
 1. the initialization would be ill-formed, or
 2. the full-expression of an invented *init-declarator* for the initialization would not be a constant expression when interpreted as a *constant-expression* ([expr.const]), or
 3. the initialization would produce a value not template-argument-equivalent ([temp.type]) to the value of the exemplar,

the program is ill-formed.

Change the example in paragraph 4:

- [...]

```
template<auto n> struct B { /* ... */ };
B<5> b1;                // OK, template parameter type is int
B<'a'> b2;              // OK, template parameter type is char
B<2.5> b3;              // OK, template parameter type is double
B<void(0)> b4;           // error: template parameter type cannot be void
```

```
template<int i> struct C { /* ... */ };
C<{ 42 }> c1; // OK
```

```
struct J1 {
    J1 *self=this;
};
B<J1{}> j1; // error: template parameter contains pointer to temporary
```

```
struct J2 {
    J2 *self=this;
    constexpr J2() {}
    constexpr J2(const J2&) {}
};
B<J2{}> j2; // error: template parameter object not
            // template-argument-equivalent to temporary exemplar
```

#[temp.type]

Change bullet (1.3):

- the exemplars ([temp.arg.nontype]) for their corresponding non-type *template-arguments* are template-argument-equivalent (see below) ~~after conversion to the type of the template-parameter~~, and